Dự án thực hành - Phân tích dữ liệu cho bộ dữ liệu chi phí bảo hiểm

Trong dự án này, bạn phải thực hiện các hoạt động phân tích trên cơ sở dữ liệu bảo hiểm sử dụng các tham số được đề cập bên dưới.

Parameter	Description	Content type
age	Age in years	integer
gender	Male or Female	integer (1 or 2)
bmi	Body mass index	float
no_of_children	Number of children	integer
smoker	Whether smoker or not	integer (0 or 1)
region	Which US region - NW, NE, SW, SE	integer (1,2,3 or 4 respectively)
charges	Annual Insurance charges in USD	float

1. Mục tiêu

Trong dự án này, bạn sẽ:

- Tải dữ liệu dưới dạng khung dữ liệu Pandas
- Làm sạch dữ liệu, loại bỏ các mục trống
- Chạy phân tích dữ liệu thăm dò (EDA) và xác định các thuộc tính ảnh hưởng nhiều nhất đến chi phí (charges)
- Phát triển các mô hình hồi quy tuyến tính một biến và đa biến để dự đoán charges
- Sử dụng hồi quy Ridge để tinh chỉnh hiệu suất của các mô hình hồi quy tuyến tính.

Thiết lập

Trong bài thực hành này, chúng ta sẽ sử dụng các thư viện sau:

- skillsnetwork để tải xuống dữ liệu
- pandas để quản lý dữ liệu.
- numpy cho các phép toán.
- sklearn cho các hàm liên quan đến học máy và đường ống học máy.
- seaborn để trực quan hóa dữ liệu.

- matplotlib cho các công cụ vẽ đồ thị bổ sung.

Các thư viện bắt buộc sau đây **không** được cài đặt sẵn trong môi trường Skills Network Labs. **Bạn sẽ cần chạy cell sau** để cài đặt chúng:

```
import piplite
await piplite.install('seaborn')
```

Nhập Thư viện Bắt buộc

Chúng tôi khuyên bạn nên nhập tất cả thư viện bắt buộc vào một nơi (tại đây):

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import cross_val_score, train_test_split
```

Tải tập dữ liệu xuống môi trường phòng thí nghiệm này

Chạy ô bên dưới để tải tập dữ liệu xuống môi trường phòng thí nghiệm này.

```
from pyodide.http import pyfetch

async def download(url, filename):
    response = await pyfetch(url)
    if response.status == 200:
        with open(filename, "wb") as f:
            f.write(await response.bytes())
```

```
filepath = 'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-DA0101EN-Coursera/medical_insurance_dataset.csv'
```

```
1  await download(filepath, "insurance.csv")
2  file_name = "insurance.csv"
```

```
1  df = pd.read_csv(file_name)
```

Lưu ý: Phiên bản thực hành này sử dụng JupyterLite, yêu cầu phải tải tập dữ liệu xuống giao diện. Trong khi thực hành trên phiên bản đã tải xuống của sổ ghi chép này trên máy cục bộ (Jupyter Anaconda), người học có thể **bỏ qua các bước trên** và sử dụng trực tiếp URL trong hàm pandas.read_csv(). Bạn có thể bỏ chú thích và chạy các câu lệnh trong ô bên dưới.

```
1  #filepath = 'https://cf-courses-data.s3.us.cloud-object-
   storage.appdomain.cloud/IBMDeveloperSkillsNetwork-DA0101EN-
   Coursera/medical_insurance_dataset.csv'
2  #df = pd.read_csv(filepath, header=None)
```

2. Nhiệm vụ 1: Nhập tập dữ liệu

Nhập tập dữ liệu vào khung dữ liệu pandas. Lưu ý rằng hiện tại không có tiêu đề nào trong tệp CSV.

In 10 hàng đầu tiên của khung dữ liệu để xác nhận việc tải thành công.

```
1  df = pd.read_csv(file_name, header = None)
2  df.head(10)
```

	0	1	2	3	4	5	6
0	19	1	27.900	0	1	3	16884.92400
1	18	2	33.770	1	0	4	1725.55230
2	28	2	33.000	3	0	4	4449.46200
3	33	2	22.705	0	0	1	21984.47061
4	32	2	28.880	0	0	1	3866.85520
5	31	1	25.740	0	?	4	3756.62160
6	46	1	33.440	1	0	4	8240.58960
7	37	1	27.740	3	0	1	7281.50560
8	37	2	29.830	2	0	2	6406.41070
9	60	1	25.840	0	0	1	28923.13692

Thêm tiêu đề vào khung dữ liệu như đã đề cập trong kịch bản dự án.

```
headers = ['age','gender','bmi','no_of_children','smoker','region','charges']
df.columns = headers
```

Bây giờ, hãy thay thế các mục **'?'** bằng giá trị **'NaN'**.

```
df.replace('?', np.nan, inplace=True)
```

3. Nhiệm vụ 2: Xử lý dữ liệu

Sử dụng dataframe.info() để xác định các cột có thông tin **'Null'** (hoặc **NaN**).

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2772 entries, 0 to 2771
Data columns (total 7 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   age             2768 non-null   object
 1   gender          2772 non-null   int64
 2   bmi             2772 non-null   float64
 3   no_of_children  2772 non-null   int64
 4   smoker          2765 non-null   object
 5   region          2772 non-null   int64
 6   charges         2772 non-null   float64
dtypes: float64(2), int64(3), object(2)
memory usage: 130.0+ KB
```

Để có thể xác định **cột nào có dữ liệu bị thiếu** bằng cách so sánh **số lượng giá trị non-null (không bị thiếu)** với **tổng số dòng** (ở đây là **2772**). Hai cột **có dữ liệu bị thiếu** là: age: thiếu 4 dòng và smoker: thiếu 7 dòng.

Xử lý dữ liệu bị thiếu:

- Đối với các thuộc tính liên tục (ví dụ: tuổi), hãy thay thế các giá trị bị thiếu bằng giá trị trung bình.
- Đối với các thuộc tính phân loại (ví dụ: người hút thuốc), hãy thay thế các giá trị bị thiếu bằng giá trị tần suất.
- Cập nhật kiểu dữ liệu của các cột tương ứng.
- Kiểm tra bản cập nhật bằng df.info().

```
avg_age = df['age'].astype('float').mean(axis=0)
df['age'].replace(np.nan, avg_age, inplace=True)

is_smoker = df['smoker'].value_counts().idxmax()
df['smoker'].replace(np.nan, is_smoker, inplace=True)

df[['age','smoker']] = df[['age','smoker']].astype('int')

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2772 entries, 0 to 2771
Data columns (total 7 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   age             2772 non-null   int32
 1   gender          2772 non-null   int64
 2   bmi             2772 non-null   float64
 3   no_of_children  2772 non-null   int64
 4   smoker          2772 non-null   int32
 5   region          2772 non-null   int64
 6   charges         2772 non-null   float64
dtypes: float64(2), int32(2), int64(3)
memory usage: 130.0 KB
```

Cũng lưu ý rằng cột charges có các giá trị dài hơn 2 chữ số thập phân. Hãy cập nhật cột **charges** sao cho tất cả **các giá trị được làm tròn đến 2 chữ số thập phân gần nhất**. Xác minh chuyển đổi bằng cách in 5 giá trị đầu tiên của khung dữ liệu đã cập nhật.

```
df[['charges']] = np.round(df[['charges']],2)
df.head(5)
```

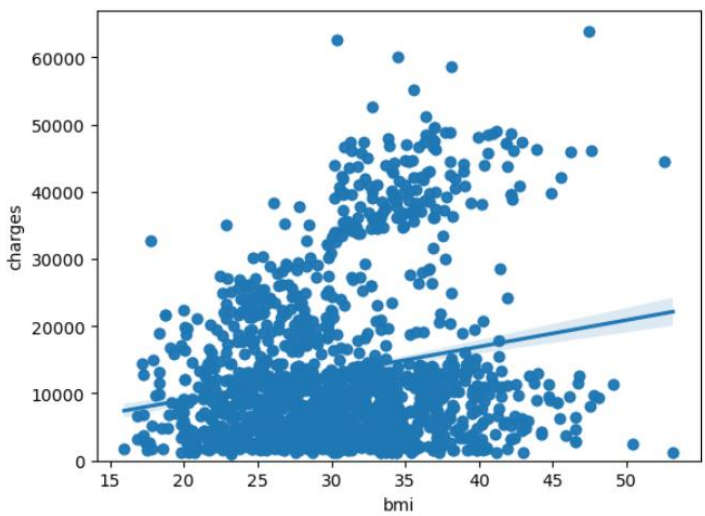
	age	gender	bmi	no_of_children	smoker	region	charges
0	19	1	27.900	0	1	3	16884.92
1	18	2	33.770	1	0	4	1725.55
2	28	2	33.000	3	0	4	4449.46
3	33	2	22.705	0	0	1	21984.47
4	32	2	28.880	0	0	1	3866.86

4. Nhiệm vụ 3: Phân tích dữ liệu khám phá (EDA)

Triển khai biểu đồ hồi quy cho các khoản charges liên quan đến BMI.

```
sns.regplot(x='bmi', y='charges', data=df)
plt.ylim(0,)
```

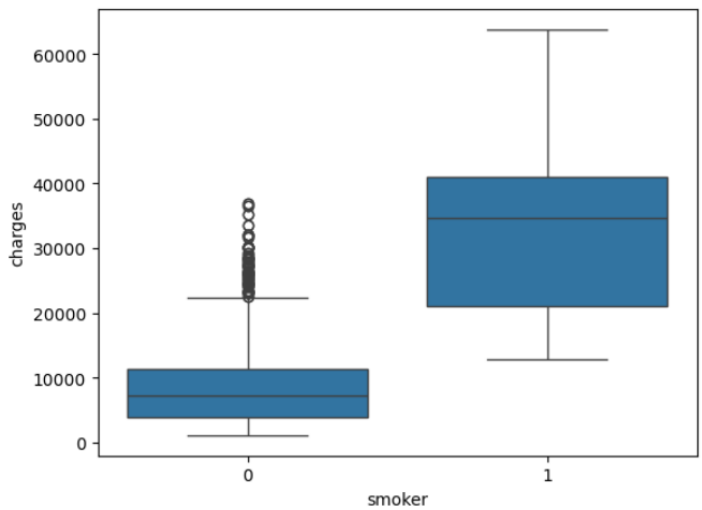
```
(0.0, 66902.85800000001)
```



Triển khai biểu đồ hộp để tính **charges** đối với người hút thuốc (**smoker**).

```
sns.boxplot(x='smoker', y='charges', data=df)
```

```
<AxesSubplot:xlabel='smoker', ylabel='charges'>
```



In ra ma trận tương quan của bộ dữ liệu.

```
df.corr()
```

	age	gender	bmi	no_of_children	smoker	region	charges
age	1.000000	-0.026046	0.113048	0.037574	-0.023286	-0.007167	0.298624
gender	-0.026046	1.000000	0.042924	0.016020	0.082326	0.022213	0.062837
bmi	0.113048	0.042924	1.000000	-0.001492	0.011489	0.271119	0.199846
no_of_children	0.037574	0.016020	-0.001492	1.000000	0.006362	-0.025717	0.066442
smoker	-0.023286	0.082326	0.011489	0.006362	1.000000	0.054077	0.788783
region	-0.007167	0.022213	0.271119	-0.025717	0.054077	1.000000	0.054058
charges	0.298624	0.062837	0.199846	0.066442	0.788783	0.054058	1.000000

5. Nhiệm vụ 4: Phát triển Mô hình

Hãy xây dựng một mô hình hồi quy tuyến tính có thể được sử dụng để dự đoán giá trị **charges**, chỉ bằng cách sử dụng thuộc tính **smoker** của tập dữ liệu. In ra hệ số $\mathbf{R^2}$ của mô hình này.

```
x_data = df[['smoker']]
y_data = df[['charges']]
lre = LinearRegression()
lre.fit(x_data, y_data)
lre.score(x_data, y_data)
```

```
0.6221791733924185
```

Hãy huấn luyện một mô hình hồi quy tuyến tính có thể được sử dụng để dự đoán giá trị **charges**, chỉ bằng cách **sử dụng tất cả các thuộc tính khác của tập dữ liệu**. In ra điểm $\mathbf{R^2}$ của mô hình này. Bạn sẽ thấy hiệu suất được cải thiện.

```
Z = df[['age', 'gender', 'bmi', 'no_of_children', 'smoker', 'region']]
lre.fit(Z, y_data)
lre.score(Z, y_data)
```

```
0.7504083820289634
```

Tạo một quy trình đào tạo sử dụng **StandardScaler()**, **PolynomialFeatures()** và **LinearRegression()** để tạo ra một mô hình có thể dự đoán giá trị charges bằng cách sử dụng tất cả các thuộc tính khác của tập dữ liệu. Hiệu suất sẽ còn được cải thiện hơn nữa.

```
Input = [('scaler', StandardScaler()), ('polynomial',
PolynomialFeatures(include_bias=False)), ('model', LinearRegression())]
pipe = Pipeline(Input)
Z = Z.astype('float')
pipe.fit(Z, y_data)
ypipe = pipe.predict(Z) # Dự đoán dựa trên pipeline đã huấn luyện
print(r2_score(y_data, ypipe))
```

```
0.8451876102882967
```

Giải thích:

- **Tại sao model.score(X, Y) lại truyền X (biến dự đoán) trước, còn r2_score(y_true, y_pred) thì lại truyền y_true (biến phụ thuộc) trước?** Vì **chúng khác nhau ở chức năng** của từng hàm!
- model.score(X, Y)**:** Đây là hàm **của mô hình học máy**, ví dụ như LinearRegression(). Bạn đưa vào:
    - X: **dữ liệu đầu vào** (biến độc lập, biến dự đoán),
    - Y: **nhãn thực tế** (biến phụ thuộc).

  Nó sẽ:
    - Tự động dự đoán y_pred = model.predict(X)
    - Tự tính r2_score(y, y_pred) và trả kết quả luôn.
- r2_score(y_true, y_pred)**:** Đây là hàm của sklearn.metrics để bạn **tự tính độ chính xác một cách thủ công** khi bạn đã có 2 vector:
    - y_true: giá trị thật (biến phụ thuộc),
    - y_pred: giá trị bạn dự đoán được.

  Vị trí này tuân theo đúng logic: **"Thật trước, Dự đoán sau"** → giống như kiểm tra bài: so sánh đáp án thật với bài bạn làm.

6. Nhiệm vụ 5: Tinh chỉnh mô hình

Chia dữ liệu thành các tập con huấn luyện và kiểm tra, giả định rằng 20% dữ liệu sẽ được dành riêng cho việc kiểm tra.

```
x_train, x_test, y_train, y_test = train_test_split(Z, y_data, test_size=0.2, random_state=1)
```

Khởi tạo một mô hình Ridge Regression với siêu tham số **α=0.1**. Huấn luyện mô hình bằng tập dữ liệu huấn luyện con. In ra điểm số $\mathbf{R^2}$ trên tập dữ liệu kiểm tra.

```
RidgeModel = Ridge(alpha=0.1)
RidgeModel.fit(x_train, y_train)
yhat = RidgeModel.predict(x_test)
r2_score(y_test, yhat)
```

```
0.676080775339254
```

Áp dụng phép biến đổi đa thức cho các tham số huấn luyện với bậc = 2. Sử dụng tập đặc trưng đã biến đổi này để huấn luyện với cùng mô hình hồi quy như trên, sử dụng tập con huấn luyện. In ra điểm **R^2** cho tập con kiểm tra.

```
Pr = PolynomialFeatures(degree=2)
x_train_pr = Pr.fit_transform(x_train)
x_test_pr = Pr.fit_transform(x_test)
RidgeModel.fit(x_train_pr, y_train)
yhat_pr = RidgeModel.predict(x_test_pr)
r2_score(y_test, yhat_pr)
```

```
0.7835631069813711
```

Xin chúc mừng! Bạn đã hoàn thành dự án này

Sự khác nhau giữa r2_score và score

1. r2_score(y_true, y_pred)

- Bạn phải **tự predict trước**:

```
yhat = RidgeModel.predict(x_test)

r2_score(y_test, yhat)
```

- Dùng tốt khi bạn cần:
    - Giúp kiểm soát cao hơn, là khi **muốn làm nhiều việc khác** với kết quả y_pred chứ không chỉ tính điểm số R^2. Dùng khi bạn cần thao tác thêm với dự đoán.ọi riêng nhiều metric (MAE, MSE, R^2,...)
    - So sánh nhiều mô hình khác nhau thủ công

    - Debug hay vẽ biểu đồ

2. model.score(X_test, y_test)

- Gọi nhanh:

```
RidgeModel.score(x_test, y_test)
```

- Nó **tự động**:
    - Dự đoán luôn (giống .predict())
    - Tính R^2
- Tiện để:
    - Kiểm tra nhanh độ chính xác mô hình
    - Dùng trong pipeline, GridSearchCV,…

🤔 **Vậy khi nào nên dùng cái nào?**

Mục đích	**Dùng cái nào?**
Chỉ cần kiểm tra nhanh R^2	.score()
Cần tự tay predict và phân tích sâu	r2_score()
Viết báo cáo với nhiều metric	r2_score()
Dùng trong cross-validation hoặc pipeline	.score() sẽ được gọi nội bộ